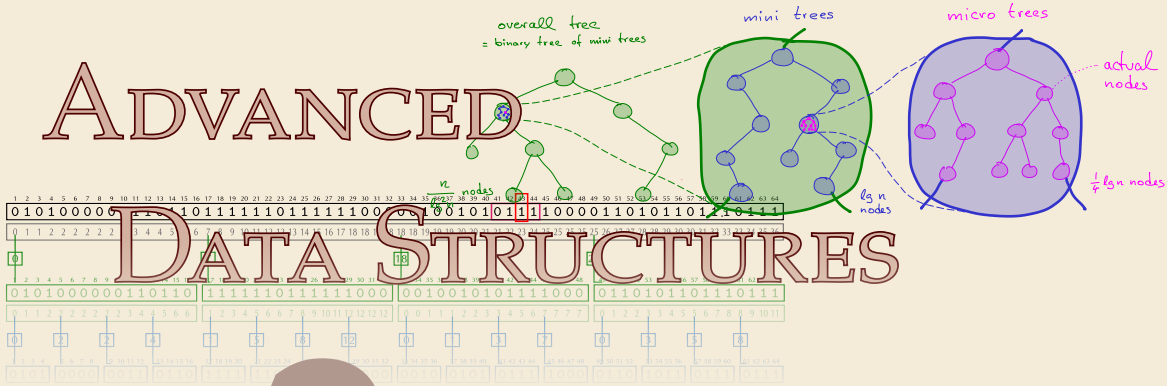


Advanced Data Structures

2 Adaptive Trees

Advanced Data Structures · Summer 2026

Prof. Dr. Sebastian Wild

CS650 (Summer 2026)
Philipps-Universität Marburg
version `2026-04-27 00:27 H`

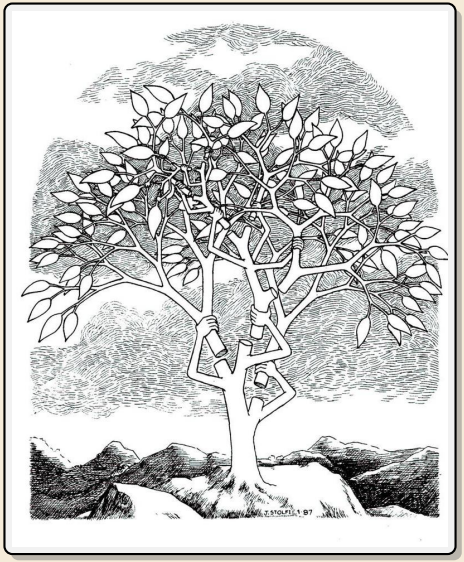
Outline

2 Adaptive Trees

2.1 Move-to-Root Heuristic
2.2 Splay Trees
2.3 Analysis of Splay Trees
2.4 Dynamic Optimality
2.5 Lazy Search Trees

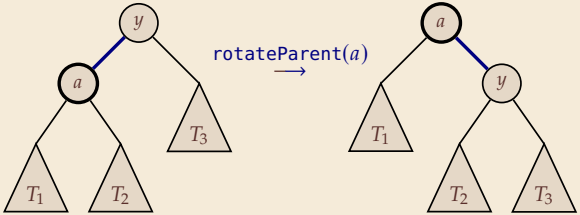
Self-Adjusting Data Structures

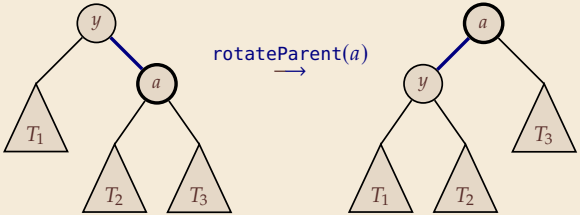
Idea: set of rules that makes data structure automatically adapt to use case



2.1 Move-to-Root Heuristic

Rotations


y
a
T1
T2
T3
rotateParent(a)
a
y
T1
T2
T3


y
a
T1
T2
T3
rotateParent(a)
a
y
T1
T2
T3

Move-to-root

Allen, Munro: *Self-Organizating Binary Search Trees*, JACM **1978**

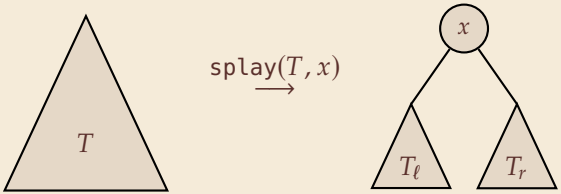
2.2 Splay Trees

Move-to-root reloaded

Sleater, Tarjan: *Self-Adjusting Binary Search Trees*, JACM **1985**

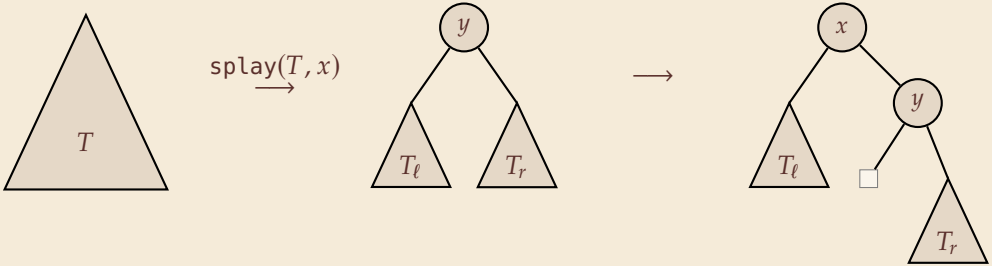
Splay

Aggressive restructuring allows to base all operations on single primitive: `splay`



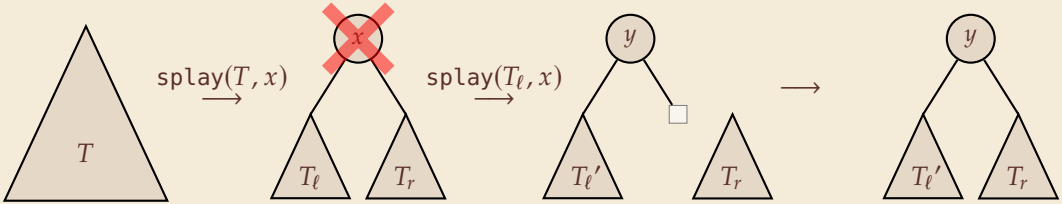
Splay – Insert

1. `splay`(x, T) $\rightsquigarrow$ root y and $T_\ell \leq y \leq T_r$ (key order)
2. If $x = y$ (key already stored) return.
3. Otherwise, w.l.o.g. $x < y$, can simply make x new root.



Splay – Delete

1. `splay`(x, T) $\rightsquigarrow$ root x (or not part of tree and we're done)
2. Remove x, left with T_ℓ and T_r
3. `splay`(x, T_ℓ) $\rightsquigarrow$ root $y = \texttt{max}(T_\ell)$ and $y.right = \texttt{null}$
4. $y.right := T_r$



Note: We've effectively built **`join`** $\rightsquigarrow$ naturally supported via `splay`.

`split` even more directly supported via `splay`: `split`(T, x): `splay`(T, x), return $(x.left, x)$

2.3 Analysis of Splay Trees

2.4 Dynamic Optimality

2.5 Lazy Search Trees